Cramming: End-to-end trained Sequence Models Defer Computation to the Last Token

Anonymous Authors[1]

Abstract

Prior work has shown that one-layer transformers trained on two-term modular arithmetic converge to a Fourier multiplication circuit. In this paper we study the behavior of a model trained on n-term modular arithmetic, and find a more complicated picture. Transformers trained to predict the final output achieve perfect accuracy on n-term addition but fail on m-term addition for $m < n$, despite m-term being a sub-problem of n-term. We call this behavior *cramming*: the model defers all computation to the final token rather than solving the problem term by term. The embedding layers exhibit the same sparse Fourier structure as in two-term models, so the bottleneck is not the input representation. Instead, for a model trained on both addition and subtraction, the residual stream is nearly identical across both sequences until the final token. Scratchpad supervision on intermediate partial sums fixes cramming and restores generalization for m-term addition when $m \leq n$.

1. Introduction

Modular arithmetic tasks have served as a useful test bed for studying how neural networks learn algorithmic problems. Interest in these tasks was sparked by Power et al. (2022), who identified the phenomenon of *grokking* where models first memorize the training data and only generalize after prolonged additional training. A line of work in mechanistic interpretability has since developed a clean picture of how transformers solve two-term modular addition: Nanda et al. (2023) showed that a one-layer transformer trained on $a + b \bmod p$ encodes its inputs in a Fourier basis and computes the sum via trigonometric identities, and Zhong et al. (2023) described two related algorithms — the "clock" and the "pizza" — that implement the same computation through circular representations. These findings have been extended to MLPs with quadratic activations (Gromov, 2023) and recurrent networks (Rangamani, 2025), other arithmetic operations (Mallinar et al., 2024), and have been studied theoretically through the lenses of circuit efficiency (Varma et al., 2023), margin maximization (Morwani et al., 2024), and training dynamics (Mohamadi et al., 2024; Mallinar et al., 2024). Together this body of work suggests that a sparse Fourier representation in the embedding layers is a generic feature of sequence models that solve modular arithmetic.

In this paper we ask whether searching for solutions with sparse Fourier spectra is sufficient to ensure out of distribution generalization, specifically in the case of arithmetic with multiple arguments. Concretely, we train one-layer transformers on n-term modular addition for $n \in \{2, 3, 4\}$ and evaluate them on m-term addition for $m \neq n$. We find that models trained to predict only the final answer do not generalize to problems they are not trained on. While it is generally expected that sequence models struggle to extrapolate to longer sequences, we also find that the models also struggle to generalize to shorter sequences. We call this behavior *cramming*: rather than computing a partial sum across the sequence, the model defers all of the computation to the final token.

Failures to generalize across sequence length have been studied extensively in the broader literature on transformers - particularly in the context of adding n-digit numbers - and a substantial body of work has attributed them to the choice of positional encoding. ALiBi Press et al. (2022), RoPE (Su et al., 2024), and Abacus (McLeish et al., 2024) embeddings have been proposed to enable models to extrapolate beyond n-digit number arithmetic. The cramming failure we identify is of a different nature. Modular addition is symmetric in its operands, so the position of each term in the sequence is not informative for the correct answer; nevertheless, end-to-end trained models bind their computation to a specific position — the final token.

Our contributions are as follows.

- We show that end-to-end trained transformers on n-term modular addition do not generalize across n, and

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

we name this failure mode *cramming*.

- We provide three pieces of mechanistic evidence — Fourier structure in the embeddings, residual stream similarity across operation sequences, and behavioral failure on compositionally derived sequences — that locate the failure downstream of the input representation and characterize it as the absence of computing the sum term by term.
- We show that scratchpad supervision (Nye et al., 2021) — training the model to predict the partial sum at each operand position rather than only the final answer, an approach that is closely related to chain-of-thought prompting (Wei et al., 2022) — is sufficient to fix cramming. The same architecture, trained in this way, generalizes across $m \leq n$ and exhibits representational divergence at the position where operations differ rather than at the end.

We release an anonymized code repository for reproducibility.[1]

2. Evidence for Cramming

We train transformers with one encoder block on n-term modular addition modulo $p = 41$ for $n \in \{2, 3, 4\}$. The transformers use 8 attention heads and has embedding dimension $d = 512$, with the MLP layer being $2\times$ as large with $d_{ff} = 1024$. The models were trained using the Adam optimizer with learning rate $1e-3$. All of our models reach near-perfect test accuracy on the specific problem they are trained on. They also contain a sparse Fourier spectrum and the Fourier multiplication circuit used to implement modular arithmetic, as described in appendix A. However, this is not sufficient to guarantee generalization out of distribution.

2.1. Models trained on n-term addition fail on m-term addition for $m < n$

We evaluate each trained model on m-term sequences for all $m \in \{2, 3, 4\}$. The results are summarized in Table 1. A model trained on n-term addition and subtraction reaches near-perfect accuracy on n-term test sequences but performs at chance on m-term sequences for $m \neq n$. This holds in both directions: a model trained on three-term arithmetic fails on two-term arithmetic, and a model trained on two-term arithmetic fails on three-term arithmetic.

The failure to generalize to more terms in the input is understandable, and a commonplace observation. However, our results find that the models do not generalize to even shorter sequences. The two-term sub-task is contained in the three-term task in the obvious sense — $a + b \bmod p$ is what the prefix of $a + b + c \bmod p$ should evaluate to — so a model that solved the addition problem term by term across the sequence would solve both. The fact that it does not is the first sign that the model is not maintaining such a partial sum.

[1]Anonymous repository: https://anonymous.4open.science/r/cramming-D22A/

Table 1. n-Term Modular Addition Performance. Each column corresponds to the sequence length n seen during training. Each row corresponds to the sequence length m during testing.

m	$n = 2$-term train	$n = 3$-term train	$n = 4$-term train
2	100.0%	0.0%	3.0%
3	0.9%	99.9%	2.2%
4	0.6%	0.3%	100.0%
5	0.7%	1.1%	2.4%
6	0.8%	1.0%	2.3%
7	1.0%	0.9%	2.3%
8	1.1%	0.9%	2.5%
9	0.9%	1.0%	2.5%
10	0.9%	0.7%	2.5%

Patched residual source	Source op	Target op
$a + b \to a - b$	0.61%	100%
$a - b \to a + b$	0.61%	100%

Table 2. Activation patching at the final (=) position. Swapping the final-token residual stream between addition and subtraction runs causes the model to produce the answer for the patched operation rather than the original one.

2.2. Activation patching localizes the computation to the final token

To test where in the sequence the operation-specific computation happens, we run an activation patching experiment. We take a model trained jointly on $a+b$ and $a-b$ (where the operation is indicated by special tokens), and for a fixed pair (a, b) we run the model on both $\langle a, +, b, = \rangle$ and $\langle a, -, b, = \rangle$. We then take the residual stream at the final (=) position from one run and substitute it into the other run before the unembedding. The unembedding is then applied as normal.

If the operation-specific computation were spread across the sequence, this swap would produce arbitrary outputs. Instead, the swap exactly inverts the prediction: patching the final-position residual from the $a - b$ run into the $a + b$ run causes the model to output $a-b \bmod p$, and vice versa. The model produces the correct answer for the *other* problem. This shows that the residual stream at the final position carries the entire operation-specific signal; everything else in the sequence does not play a role in the computation of the output.

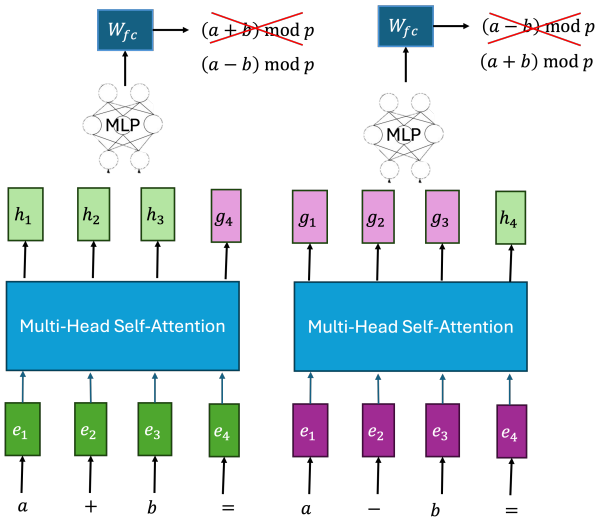

Figure 1. Activation patching at the final (=) position. We run the model on an addition prompt (left, green) and a subtraction prompt (right, purple), then swap the post-attention residual stream at the final token between the two runs ($h_4 \leftrightarrow g_4$) before the MLP. The patched residual is then passed through the remaining MLP and unembedding layers. The model produces the answer corresponding to the patched operation rather than the original one (red $\times$).

2.3. Cosine similarity confirms identical representations until the final token

The activation patching result implies that the residual streams at non-final positions are similar across the two operation sequences. We verify this directly. For the same pairs (a, b), we compute the cosine similarity between the residual stream of $\langle a, +, b, = \rangle$ and $\langle a, -, b, = \rangle$ at each token position, after the transformer block. Figure 2 shows the result. At the a and b token positions the cosine similarity is essentially 1 (while at the operation token the similarity drops since we use different op tokens). The similarity drops only at the $=$ position, where it falls close to 0.

The model is therefore not computing anything operation-specific at the operand positions; it is passing the operand representations through unchanged and producing the answer at the $=$ position alone. This is what we mean by *cramming*: the entire computation is crammed into the final token rather than being computed term by term.

2.4. Mixed-operation sequences expose the failure of compositionality

The strongest test of whether a model has learned a compositional algorithm is to ask it to compose operations in a way it has not seen during training. We train a model on three-term sequences of the form $a + b + c$ and $a - b - c$ — that is, sequences in which the two operations are uniform — and evaluate on $a + b - c$ and $a - b + c$, in which the operations are mixed.

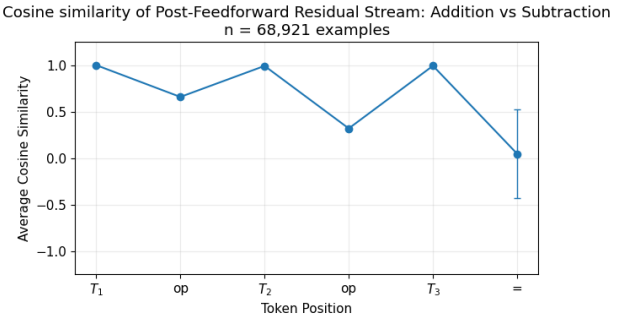

Figure 2. Cosine similarity between post-encoder block residual streams for addition and subtraction sequences in three-term models. The representations are nearly identical at operand positions, while the similarity drops sharply at the final (=) position, indicating that the operation-specific computation is localized to the final token.

Operation pattern	T_1 (a)	T_2 (b)	T_3 (c)	$=$
$(+, +)$	4.88%	2.50%	2.40%	100.00%
$(-, -)$	4.88%	2.38%	2.19%	100.00%
$(+, -)$	4.88%	2.50%	2.53%	2.10%
$(-, +)$	4.88%	2.38%	2.60%	2.00%

Table 3. Per-position accuracy for models trained on uniform three-term operation sequences and evaluated on both uniform and mixed-operation patterns. Models are perfect at the final (=) position on in-distribution patterns $(+, +)$ and $(-, -)$, but collapse to near-chance accuracy on unseen mixed patterns $(+, -)$ and $(-, +)$.

Table 3 reports the accuracy at each token position. On uniform sequences (training distribution) the model is perfect. On mixed sequences the accuracy at the first two operand positions is unchanged at 100%, since these only depend on the prefix. But at the $=$ position the accuracy collapses to $\approx$ 2% — chance. The model has learned to compute $a+b+c$ and $a-b-c$ as two separate end-to-end functions, and has not learned to apply $+$ or $-$ as operations in their own right. If it had, mixing them at unseen positions would achieve larger than chance accuracy.

3. Scratchpad Supervision Fixes Cramming

We train the same architecture as in Section 2, but with supervision for every intermediate step rather than only at the final one. For an n-term sequence $\langle a_1, \mathrm{op}_1, a_2, \mathrm{op}_2, \ldots, a_n, = \rangle$, the target at position i is the partial sum $s_i = a_1 \oplus_1 a_2 \oplus_2 \cdots \oplus_{i-1} a_i \bmod p$, where $\oplus_j$ is the operation op_j. The model is trained autoregressively with a cross-entropy loss summed over all positions.

The end-to-end objective penalizes only the prediction at

Train on n terms	Test $m = 2$	Test $m = 3$	Test $m = 4$
$n = 2$	100%	0.9%	0.6%
$n = 3$	100%	100%	0.3%
$n = 4$	100%	100%	100%

Table 4. Generalization across sequence lengths for models trained with scratchpad supervision. Reported accuracies are measured at the operand positions. Training on n-term addition yields perfect generalization to all shorter sequence lengths $m \leq n$, including settings never observed during training.

the final token,

$$\mathcal{L}_{\text{e2e}} = -\log p_\theta(s_n \mid \langle a_1, \text{op}_1, \ldots, a_n, = \rangle),$$

while the scratchpad objective penalizes the partial sum at every position,

$$\mathcal{L}_{\text{scratch}} = -\sum_{i=2}^{n} \log p_\theta(s_i \mid \langle a_1, \text{op}_1, \ldots, a_i \rangle).$$

The first objective leaves the intermediate predictions unconstrained; the second forces the model to maintain a running partial sum that is linearly decodable at every position.

3.1. Generalization across n

Models trained with scratchpad supervision on n-term sequences solve m-term sequences for all $m \leq n$. A model trained on three-term addition reaches near-perfect accuracy on two-term sequences without ever seeing them during training, in contrast to the chance-level performance of the end-to-end model in Section 2.1.

3.2. Representations diverge where the operations differ

The cosine similarity diagnostic from Section 2.3 gives a mechanistic view of what the scratchpad model is doing differently. We train a scratchpad model jointly on $a+b+c$ and $a-b-c$, and for fixed (a, b, c) compute the cosine similarity between the post-block residual streams of the two sequences at each position. Figure 3 shows the result. The similarity is ≈ 1 at the a position, where the two sequences are identical, and drops sharply at the position where the first diverging operator is fed in. It does not stay near 1 across the operand positions and only drop at the end, as in the end-to-end model.

We read these results as showing that scratchpad supervision is one way to induce a model that learns term-by term. Whether this is the only way to achieve the same effect — for example, whether an end-to-end model with a different inductive bias on the residual stream would converge to the same solution — and whether the underlying issue is one of loss landscape rather than representational capacity, are questions we leave to future work.

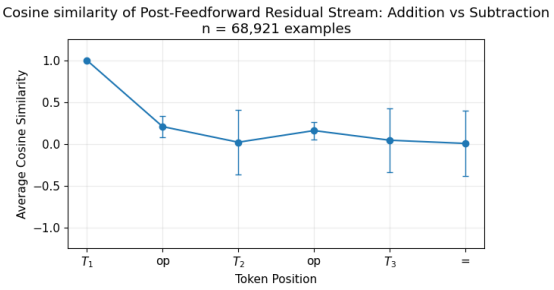

Figure 3. Cosine similarity between post-block residual streams for the scratchpad model on $a+b+c$ and $a-b-c$, measured position-by-position for fixed (a, b, c). Unlike the end-to-end model in Figure 2, where representations remain similar until the $=$ token, the scratchpad model's representations diverge immediately when the first differing operator is encountered.

4. Discussion

Our results split the picture of how transformers solve modular arithmetic in two. The input-side geometry is fine: n-term models exhibit the same sparse Fourier embeddings as their two-term counterparts. However tracking the emergence of this circuit alone is insufficient to guarantee other notions of generalization. We present evidence for the hypothesis that sequence models trained to predict only the final output cram all the computation into the last token. Scratchpad supervision fixes this by constraining how the model computes, not just what it computes.

Limitations: Our experiments are limited to one-layer transformers at small p and n ≤ 4, and our mechanistic evidence cannot fully distinguish a model with no intermediate ledger from one that maintains but does not read out such a representation, nor can it distinguish scratchpad teaching a ledger circuit from scratchpad reducing n-term to repeated two-term invocations. Whether deferring computation to the last position before a target token is a general failure mode of end-to-end trained sequence models is a question we leave to future work.

References

Gromov, A. (2023). Grokking modular arithmetic. *arXiv preprint arXiv:2301.02679*.

Mallinar, N., Beaglehole, D., Zhu, L., Radhakrishnan, A., Pandit, P., and Belkin, M. (2024). Emergence in non-neural models: grokking modular arithmetic via average gradient outer product. *arXiv preprint arXiv:2407.20199*.

McLeish, S., Bansal, A., Stein, A., Jain, N., Kirchenbauer, J., Bartoldson, B. R., Kailkhura, B., Bhatele, A., Geiping, J., Schwarzschild, A., et al. (2024). Transformers can do arithmetic with the right embeddings. *Advances in Neural Information Processing Systems*, 37:108012–108041.

Mohamadi, M. A., Li, Z., Wu, L., and Sutherland, D. J. (2024). Why do you grok? a theoretical analysis of grokking modular addition. *arXiv preprint arXiv:2407.12332*.

Morwani, D., Edelman, B. L., Oncescu, C.-A., Zhao, R., and Kakade, S. M. (2024). Feature emergence via margin maximization: case studies in algebraic tasks. In *The Twelfth International Conference on Learning Representations*.

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. (2023). Progress measures for grokking via mechanistic interpretability. *arXiv preprint arXiv:2301.05217*.

Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C., and Odena, A. (2021). Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*.

Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. (2022). Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*.

Press, O., Smith, N., and Lewis, M. (2022). Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*.

Rangamani, A. (2025). Low rank and sparse fourier structure in recurrent networks trained on modular addition. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. (2024). Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.

Varma, V., Shah, R., Kenton, Z., Kramár, J., and Kumar, R. (2023). Explaining grokking through circuit efficiency. *arXiv preprint arXiv:2309.02390*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Zhong, Z., Liu, Z., Tegmark, M., and Andreas, J. (2023). The clock and the pizza: Two stories in mechanistic explanation of neural networks. *Advances in Neural Information Processing Systems*, 36.

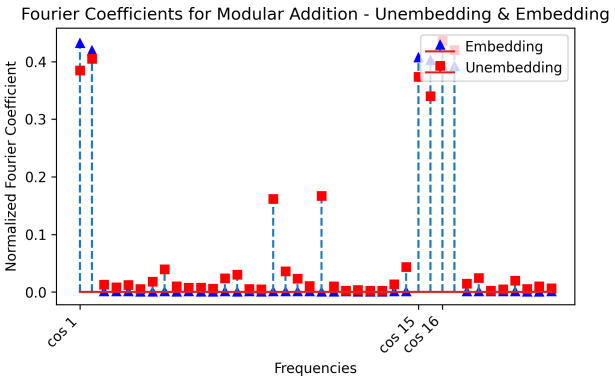

Figure 4. Normalized Fourier coefficients of the embedding and unembedding for a transformer trained on three-term modular addition modulo $p = 41$. The spectrum is sparse, similar to two-term models (Nanda et al., 2023), and the embedding and unembedding share the same Fourier support.

A. Sparse Fourier structure emerges but does not imply length generalization

Nanda et al. (2023) showed that a one-layer transformer trained on two-term modular addition learns embeddings whose rows are concentrated on a small number of Fourier basis vectors, and that the unembedding shares this support. We observe the same structure in our n-term models for $n \in \{3, 4\}$. Figure 4 shows the normalized Fourier coefficients of the embedding and unembedding for a model trained on three-term addition; the spectrum is concentrated on a handful of frequencies, and the embedding and unembedding share the same support. In this respect, the three-term and four-term models share model properties with their two-term counterparts. This observation rules out one possible explanation for the generalization failures we describe below. If the embeddings were not in the right Fourier basis, we might not expect length generalization to happen. That does not seem to be the case here.